

Patrones de Diseño Proyecto

Durante el Proyecto se aplicaron cuatro patrones de diseño fundamentales para asegurar alta cohesión, bajo acoplamiento y escalabilidad.

A. Patrones Creacionales

Singleton (Clase Tienda):

Propósito: Garantizar que exista una única instancia del inventario y presupuesto compartida en toda la aplicación, proporcionando un punto de acceso global.

Implementación: Se privatizó el constructor de Tienda y se expuso el método estático getInstance(). Esto evita que diferentes partes del código (como el Controlador o los Comandos) creen múltiples tiendas con presupuestos desincronizados.

Factory Method (Clase MascotaFactory):

Propósito: Proporcionar una interfaz para crear objetos en una superclase, permitiendo que las subclasses alteren el tipo de objetos que se crearán.

Implementación: Se utiliza al momento de adoptar. La Tienda no necesita saber si está instanciando un Perro o un Gato; simplemente le pasa el tipo como un String a la fábrica y esta retorna la instancia polimórfica correcta, centralizando la lógica de creación.

B. Patrones de Comportamiento

Observer (Interfaz EstadoObservador):

Propósito: Definir un mecanismo de suscripción para notificar a múltiples objetos sobre cualquier evento que le suceda al objeto que están observando.

Implementación: VentanaPrincipal implementa esta interfaz y se "suscribe" a la Mascota activa. Así, cuando el GameLoop desgasta el hambre o la felicidad, la mascota notifica automáticamente a la vista para que se repinte. Esto eliminó la necesidad de forzar el refresco de la UI manualmente desde el Controlador, logrando un desacoplamiento total entre el Modelo y la Vista.

Command (Interfaz AccionCuidado y sus implementaciones)

Propósito: Convertir una solicitud en un objeto independiente que contiene toda la información sobre la solicitud.

Implementación: Los botones de acción (Alimentar, Jugar, Curar, Limpiar) ya no ejecutan lógica condicional dentro del Controlador Juego. En su lugar, instancian comandos (ComandoAlimentar, etc.) que encapsulan el costo monetario, el consumo de suministro y el efecto sobre la mascota. Si a futuro se requiere añadir una acción como "Pasear", solo se crea una nueva clase Comando sin tocar la arquitectura existente.